CS 3354.011 Software Engineering
Term Project Deliverable #2 Report
Team 10 – The CRUD Crew

Contact Manager Software

GitHub Repository URL: https://github.com/AaditAggarwal/3354-TheCRUDCrew

2. Delegation of Tasks

- Aadit Aggarwal:
  - I created sequence diagrams for the use case of searching for a contact and for the use case of exporting contacts to a file.
  - I created the list of functional requirements for our software.
  - I helped develop prototype for our proposed application.
  - I created slides for searching a contact and functional requirements for the software.
- Rishab Gaddam:
  - I created sequence diagram for the use case of maintaining contacts, which included adding, deleting, and viewing contacts.
  - I described and explained which software process model we have employed in our project.
  - I helped develop prototype for our proposed application and created the test case for one unit of our software.
  - I created slides for maintaining contacts, prototype for our application, and the test case.
- Janette Duran:
  - I created sequence diagram for the use case of editing information about a contact, which included updating the name, phone number, and the photo label.
  - I detailed the assumptions we have made regarding this project and I addressed feedback that was provided in our project proposal.
  - I compared our work with software that have similar designs in this domain.
  - I developed theme of the slideshow and created slides for comparison of work with similar design and for the objectives.
- Necheer Mirza:
  - I created sequence diagrams for the use case of blacklisting a contact and for the use case of loading contacts from a file.
  - I created the diagram for the architectural design pattern that we have selected for this project.
  - I wrote conclusion and detailed changes that we made throughout project.
  - I created slides for architectural design, challenges, and conclusion.
- Naksh Jajoo:
  - I created sequence diagrams for the use case of making a phone call and for the use case of sending an SMS message.
  - I created the list of non-functional requirements for our software.

- I created the project scheduling and the estimation for cost, effort, and pricing.
- I created slides for project timeline, non-functional requirements, and cost estimation.

- Prem Patel:
  - I created sequence diagrams for the use case of managing contact groups and for the use case of sorting contacts.
  - I created the class diagram for this project.
  - I created the use case diagram for this project.
  - I cited references of sources used and created slides for class diagram and use case diagram.

3. Deliverable #1 Submission

a. Brief Software Summary

The Contact Manager Software system will allow users to manage and organize contact information easily and efficiently. Our system will allow users to add, view, edit, and delete contacts. Each contact created will have specific information associated with it, including phone number, name, and photo label. Other basic functionality of the software includes being able to get connected to systems that send SMS and make phone calls, search for a particular contact by name, and block phone calls and SMS for selected contact. System also supports contact groups. The user can create contact groups, and each group can have multiple contacts inside it. Lastly, contacts can be loaded into the software from a file, and they can also be exported from the software to a file.

b. Assumptions

It is assumed that the Contact Manager Software is built to be a mobile application, therefore, it is not supported on any non-mobile devices, which include desktop computers, laptops, and other devices. Also, the application is designed to support a single user per device, so different users cannot login and access their accounts on the same device. The user who owns the device on which the application is downloaded will be the only user and account that is supported. Lastly, it is assumed that the user is connected to a reliable service that allows making phone calls and sending SMS messages. The Contact Manager System only connects users to these external services that allow them to make phone calls and send SMS messages when they initiate those actions, it does not perform those services itself. Therefore, the Contact Manager System is not responsible for any issues encountered while performing these services, it is only responsible for providing users with access to the appropriate external systems.

c. Address Feedback from Project Proposal

The feedback we received on the final proposal was “well done,” we acknowledged this as positive feedback and appreciate the commentary. Ultimately, there was no additional feedback provided to us, so we will continue to ensure our project will meet the expected

standards and strive to show further improvement as we continue moving forward. Other than that, after carefully reviewing the instructions, we decided to slightly modify our task delegation from what we initially planned during the project proposal. Instead of each member contributing to the use case diagram, only one individual developed it. Furthermore, some individuals developed different sequence diagrams than what was initially proposed to even out the workload. In short, we have ensured that this deliverable includes all the required sections that each contain all the necessary content to meet the given requirements.

d. Describe Software Process Model

For this project, we have decided to employ the incremental process model. We chose this model because since this is our first software project, building this software will require us to first produce a basic working software and then add features and functionality as we gain more knowledge. It is also quite difficult for us to fully visualize the perfect layout and interface we should implement, so we will have to first make a basic software, experiment with the layout, observe drawbacks in that layout, and then develop better increments that address those drawbacks. As we experiment more with the design, we will also discover ways to improve usability, which will demand further increments. The waterfall model is not appropriate for our software because all the requirements are not fully clear and nor do we believe we can fully specify them before writing a single line of code. It is not practical to assume that once we finish implementation and move on to testing that we will not need to go back and address any new functionalities that may come to mind later. Therefore, because our requirements are not fully clear and we anticipate developing the software in increments, we have decided to employ the incremental process model.

e. Software Requirements

i. Functional Requirements

1. The system shall allow users to add a contact entry, where the contact entry stores a name, one or more phone numbers, and an optional photo label.
2. The system shall allow users to delete an existing contact entry.
3. The system shall allow users to add a contact to blacklist, which will block all incoming SMS messages and phone calls from that contact.
4. The system shall allow users to start a phone call directly from a contact view without having to exit out of it.
5. The system shall allow users to send an SMS directly from contact view.
6. The system shall allow users to create contact groups for contacts in their existing contact list.
7. The system shall allow users to export the current contact list to an external file in a supported format and also allow loading of such files to import contacts.

ii. Non-functional Requirements

1. Product Requirements
   a. Efficiency Requirements - The system shall perform operations effectively and respond quickly to user actions.
      i. Performance Requirements
         1. Contact lists, search operations and details shall load and be displayed within 2 seconds.
         1. Phone calls shall move to the phone call page view within 3 seconds of user request.
         2. SMS messages shall move to SMS page view within 3 seconds.
      ii. Space Requirements
         1. The system shall store contact data efficiently to support at least 500 contacts. without significant storage overhead.
         2. Contact files and images shall be compressed or optimized to reduce storage usage.
   b. Usability Requirements
      i. The interface shall be simple and easy to navigate and users shall be able to perform common tasks with minimal steps.
      ii. The user shall be able to utilize all provided functions in the software after observing 15 minutes of demonstration.
      iii. The system shall provide clear feedback and error messages that limit error clicks to no more than 5%.
   c. Dependability Requirements
      i. The system shall be available to perform all functionality within 5 seconds once user opens the application.
      ii. The system shall not lose any data that has been inputted and stored into it.
   d. Security Requirements
      i. Contact data shall be stored securely.
      ii. Occurrence of restricted data being accessible by or being visible to the user shall be 0%.
      iii. User input shall be validated to prevent invalid or malicious data.
2. Organizational Requirements
   a. Environmental Requirements
      i. The system shall operate on common operating systems such as Android and iOS.
      ii. The application shall run on standard hardware without requiring specialized hardware.
   b. Operational Requirements
      i. The system shall allow exporting and importing contact files for backup and recovery.
      ii. The application shall maintain stable operation during normal use.

c. Development Requirements

i. The system shall follow the Model-View-Controller (MVC) architecture.

ii. The codebase shall be modular to allow easy maintenance and updates and the system should be properly documented.

3. External Requirements

a. Regulatory Requirements

i. The system shall follow standard software development practices and data management guidelines.

b. Ethical Requirements

i. The system shall ensure the user contact data is handled responsibly and not shared without permission.

c. Legislative Requirements

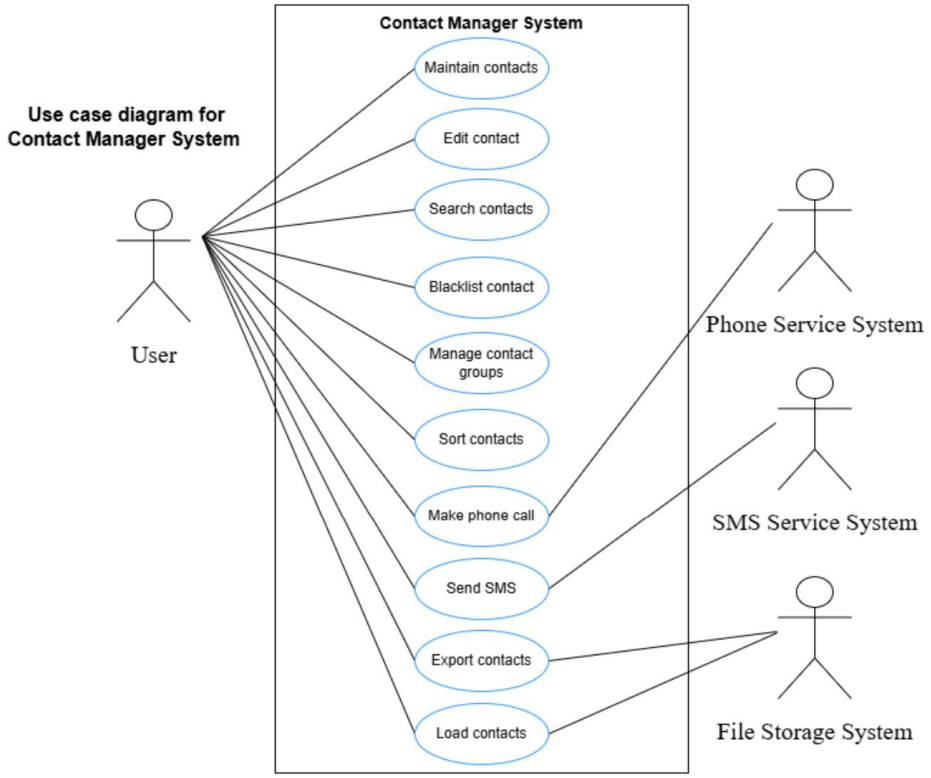
i. Accounting Requirements

1. Any exported contact data shall maintain accurate and complete information.
2. All information in exported data shall match completely with information present in data in the software.

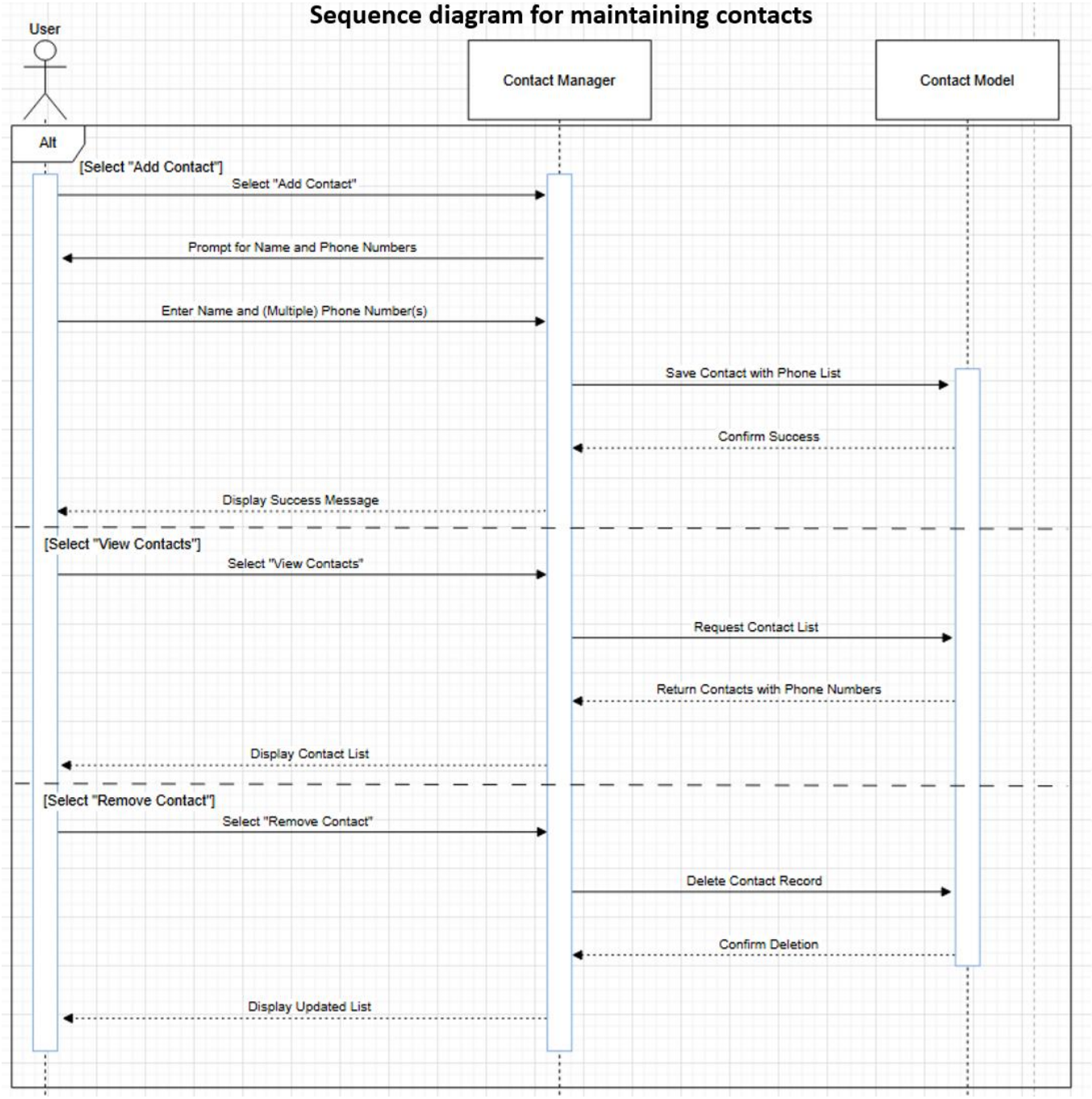
ii. Safety/Security Requirements

1. The system shall protect user data from unauthorized access or modification.
2. The system shall notify users if system errors or failures occur within 5 seconds of the error or failure occurring.

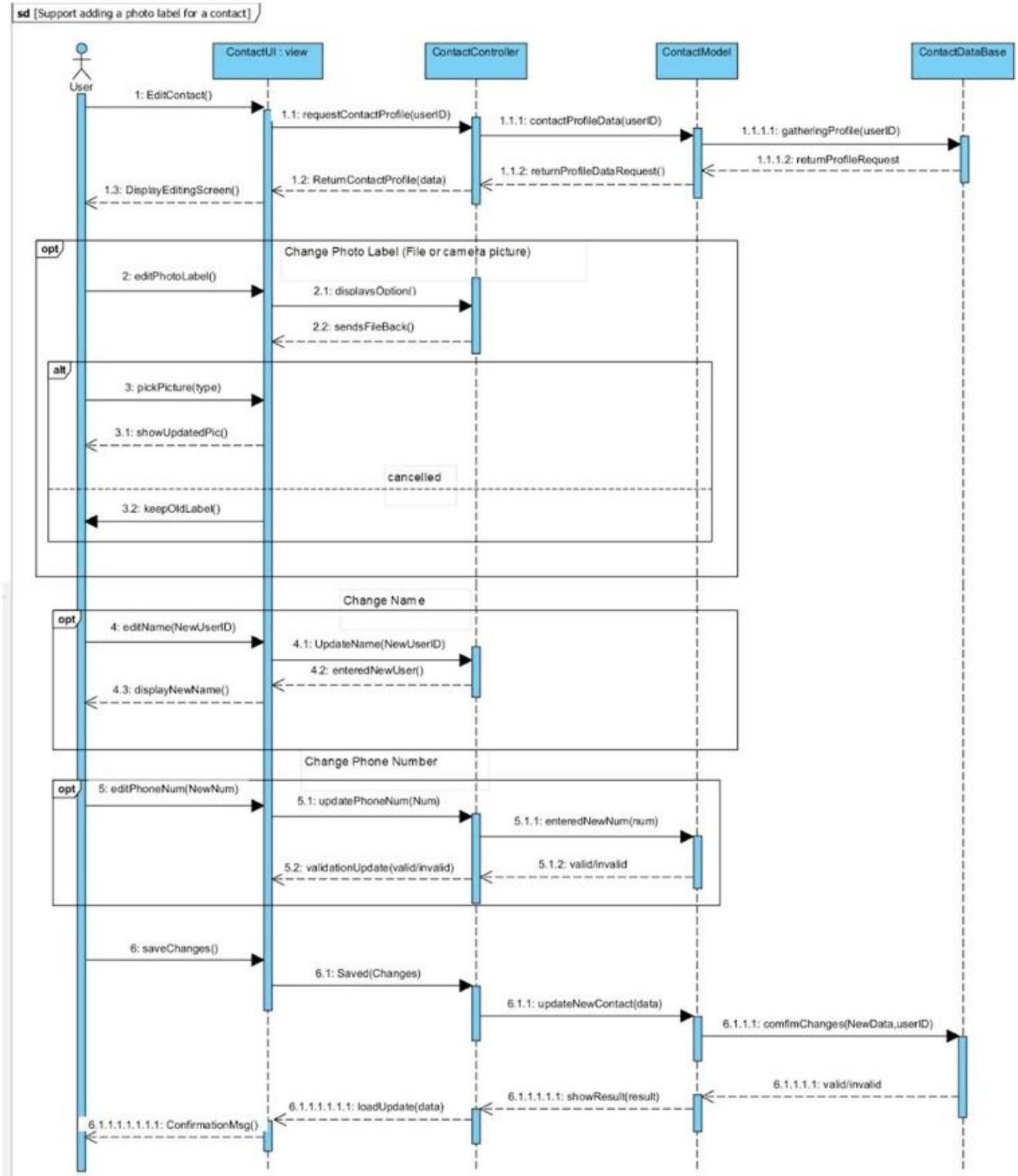
f. Use Case Diagram



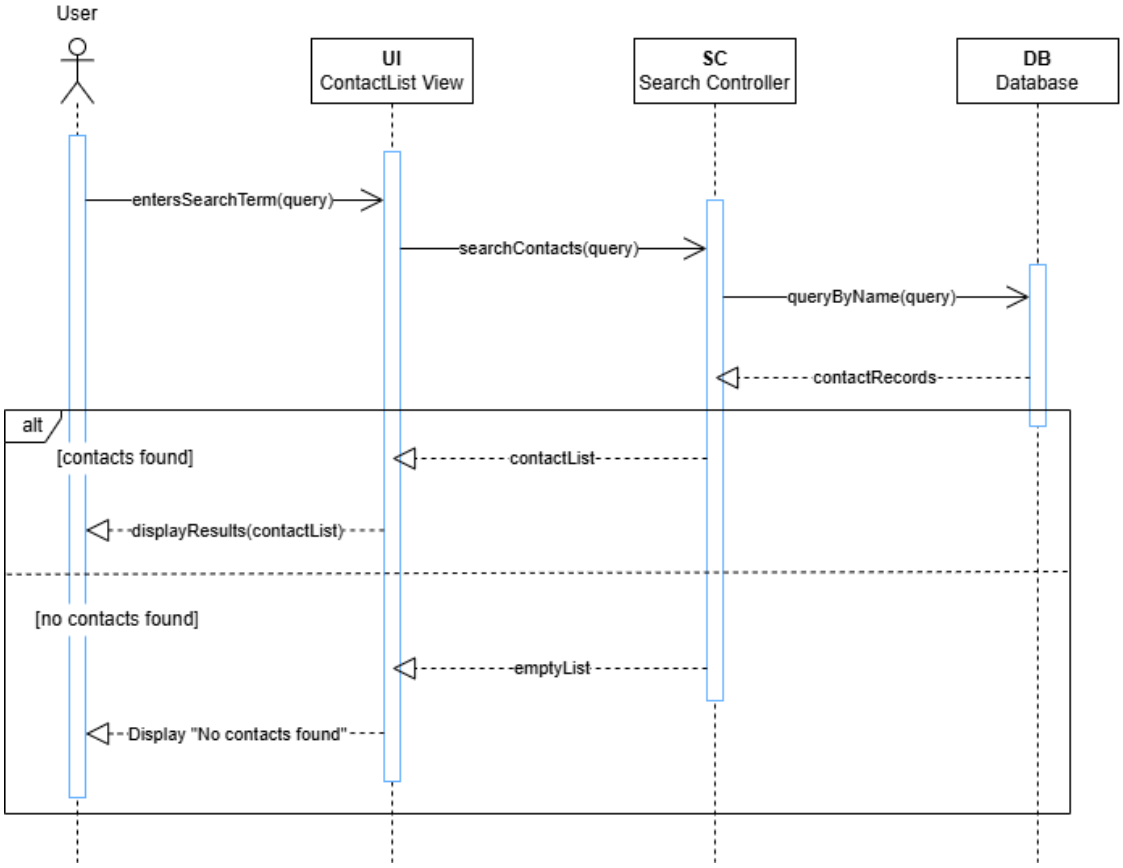
g. Sequence Diagrams for each Use Case



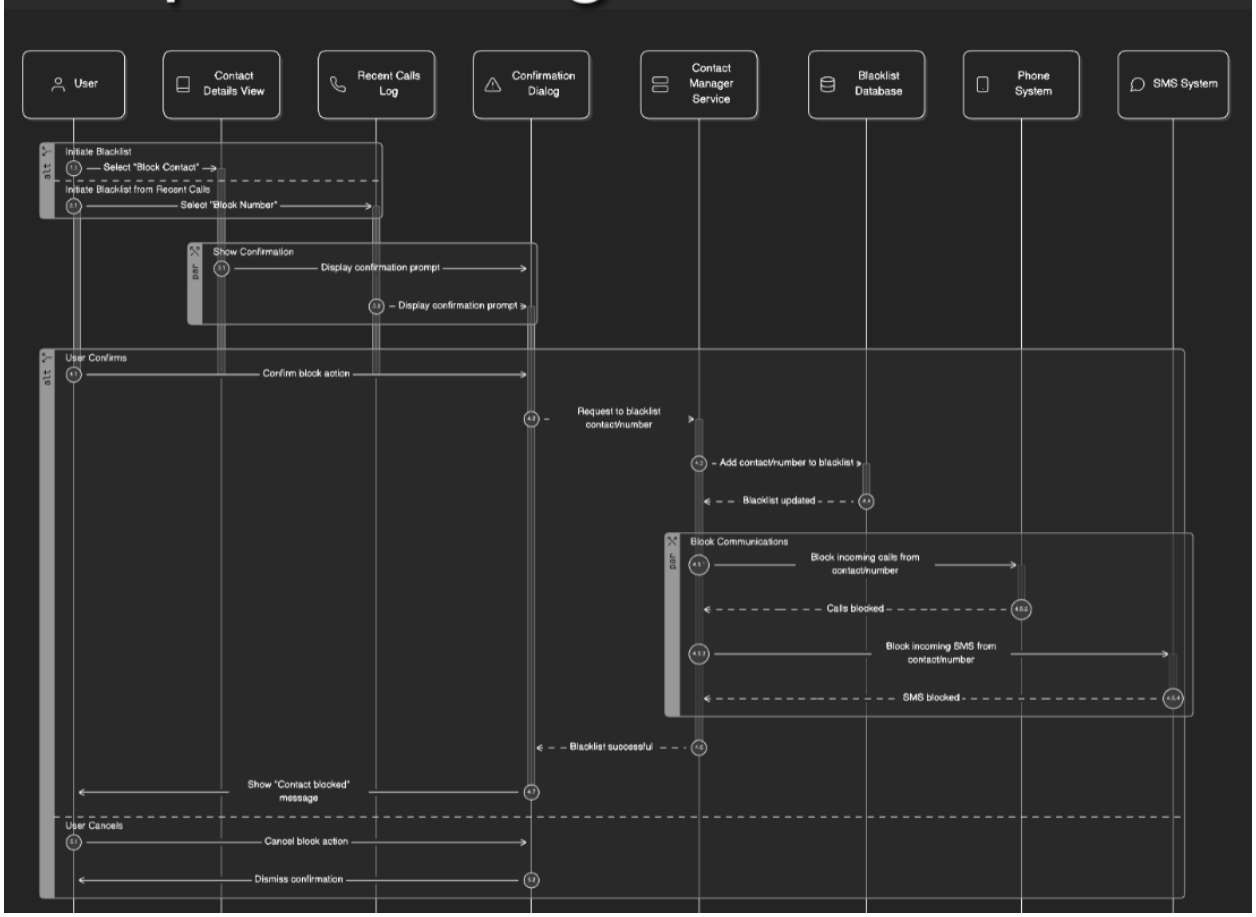
Sequence diagram for editing a contact



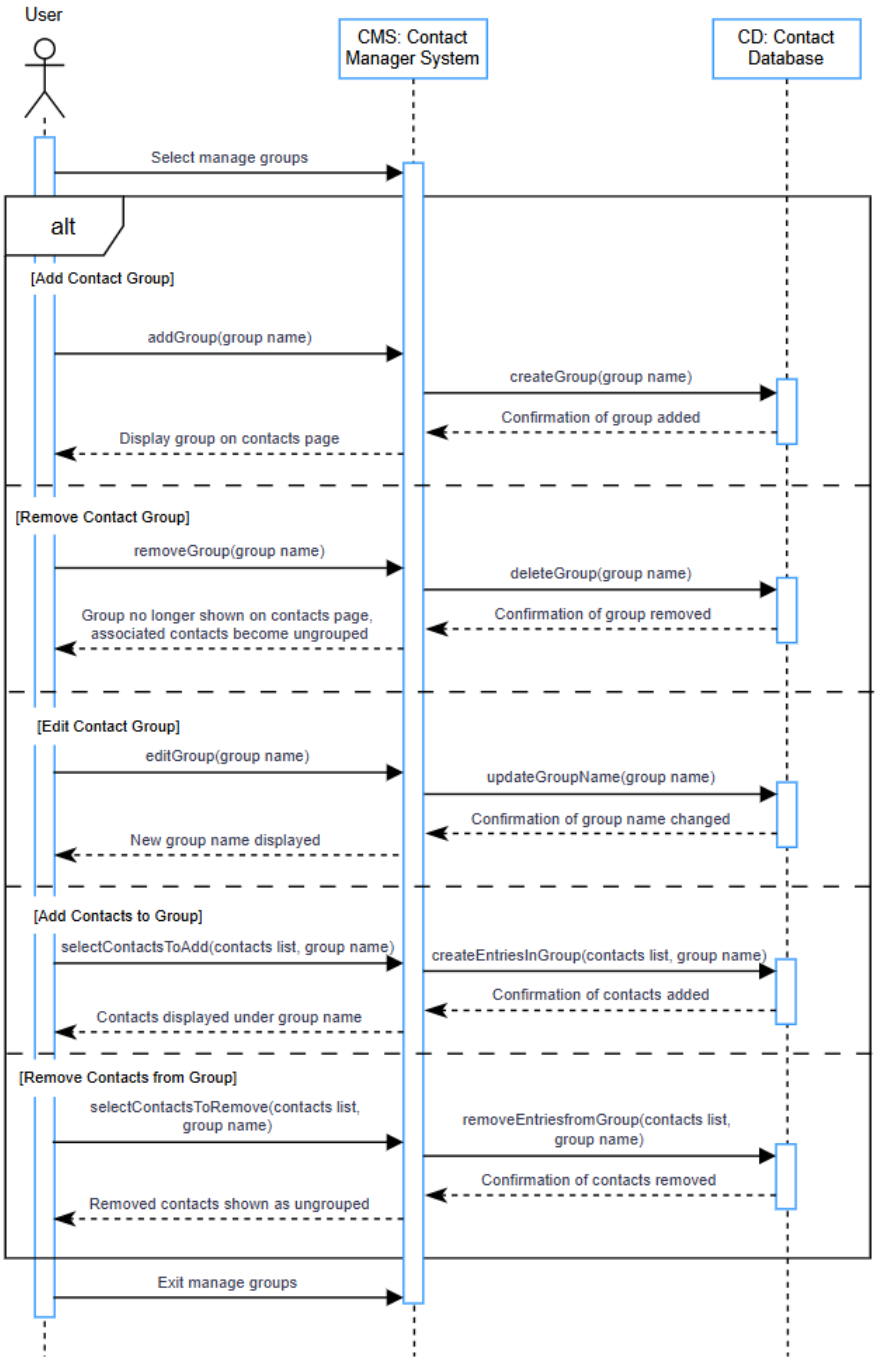
Sequential Diagram for Search Query



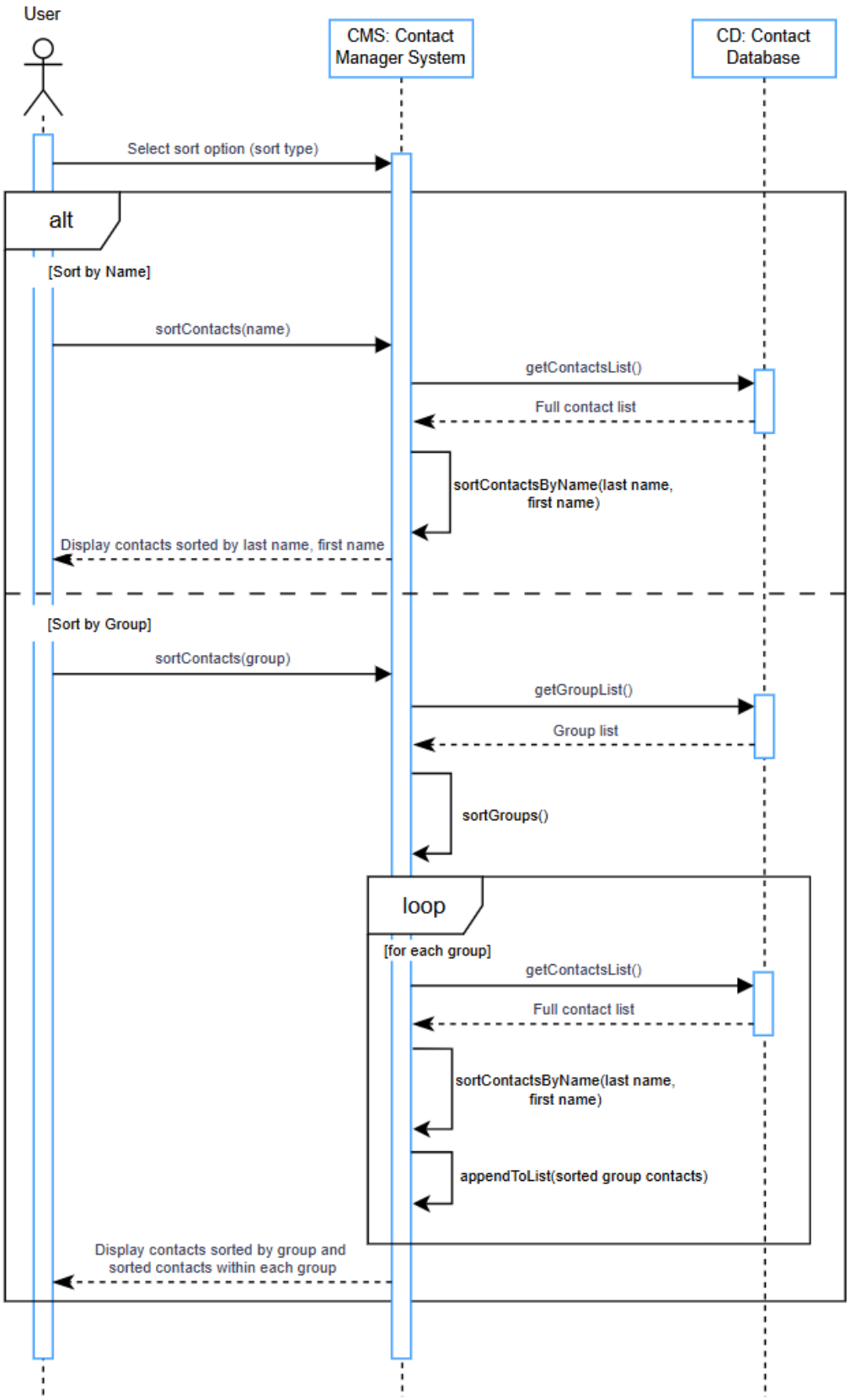
Sequence Diagram for Blocklist



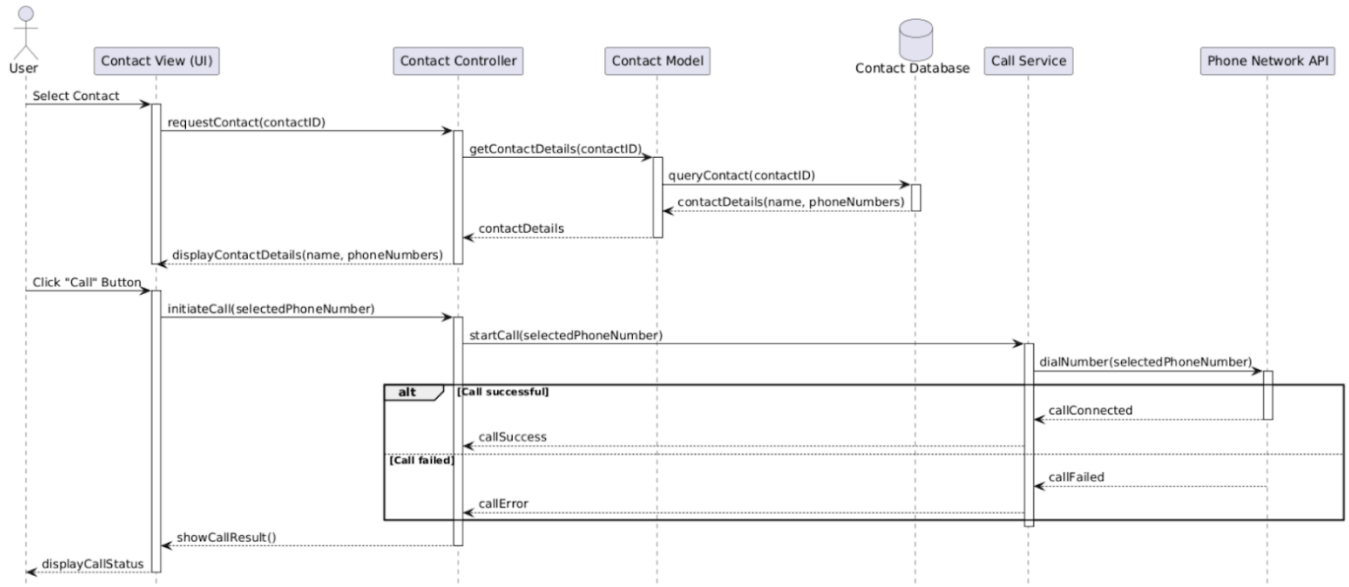
Sequence diagram for Managing Contact Groups



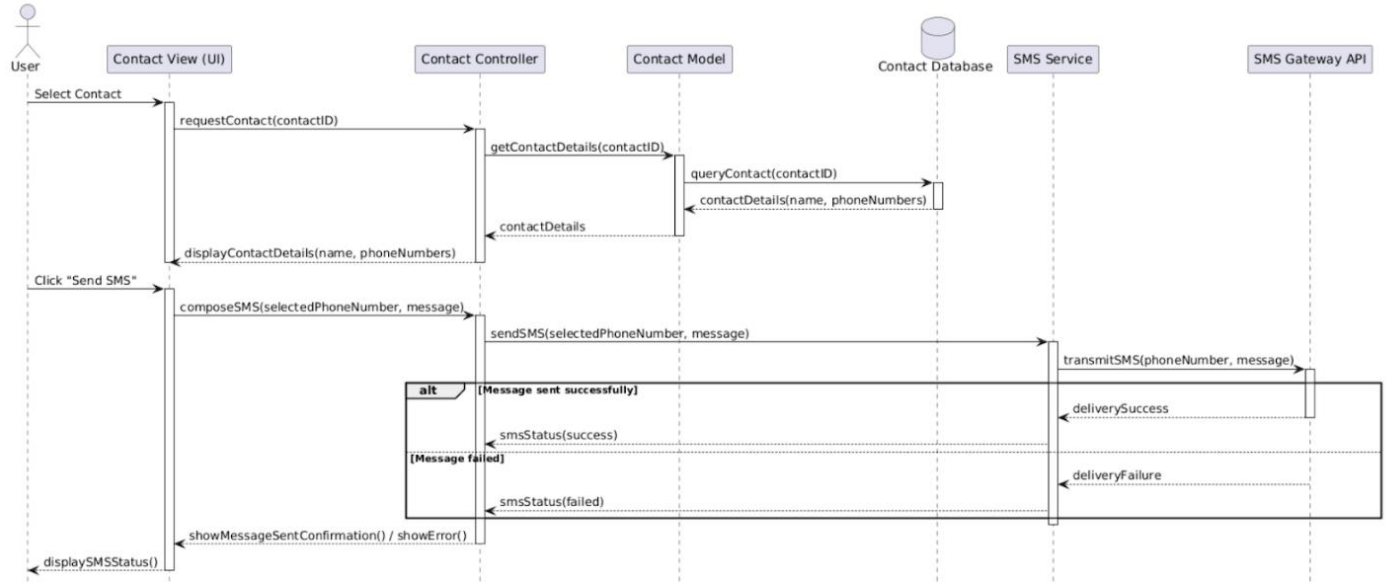
Sequence diagram for Sorting Contacts



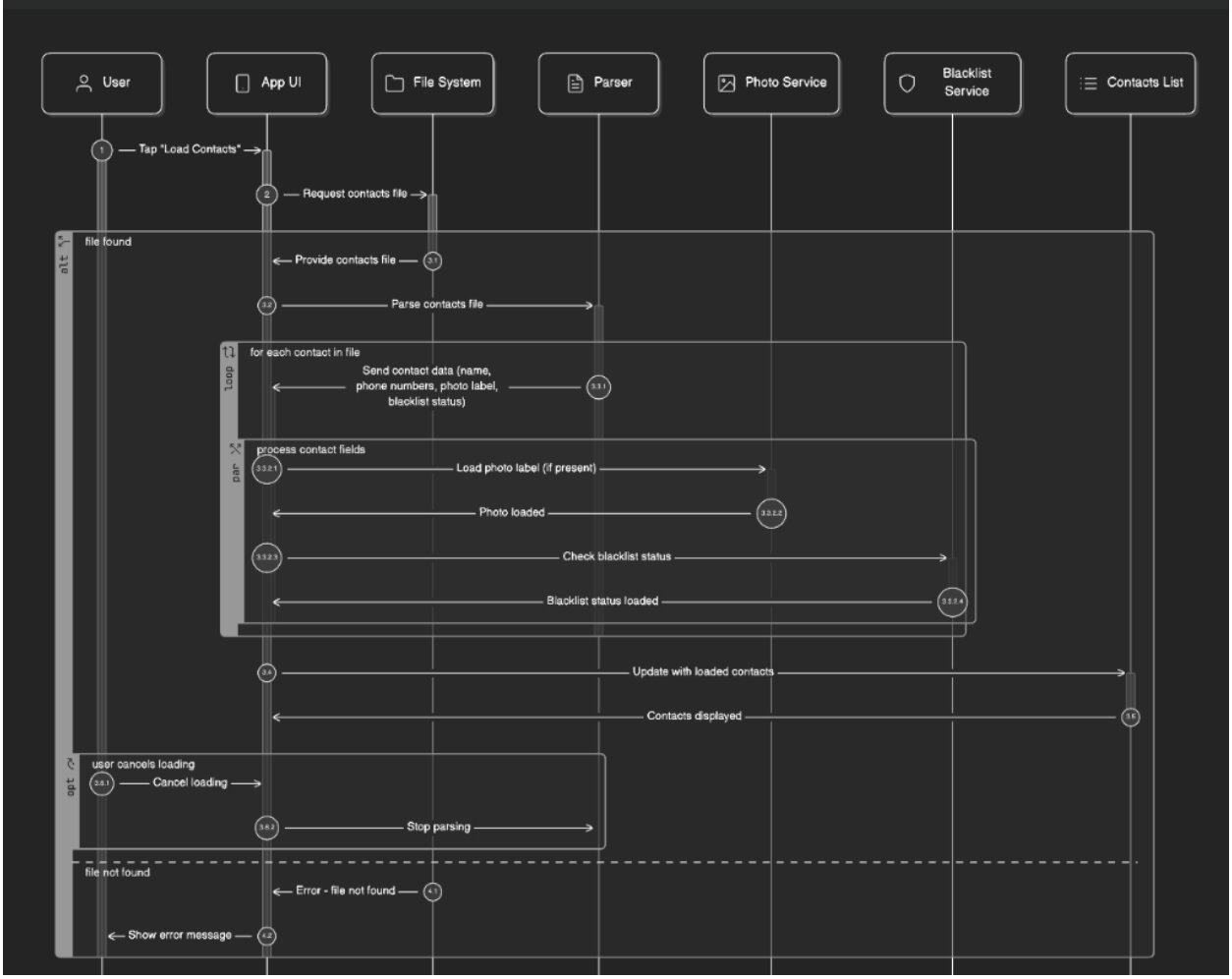
Sequence Diagram - Make Phone Call



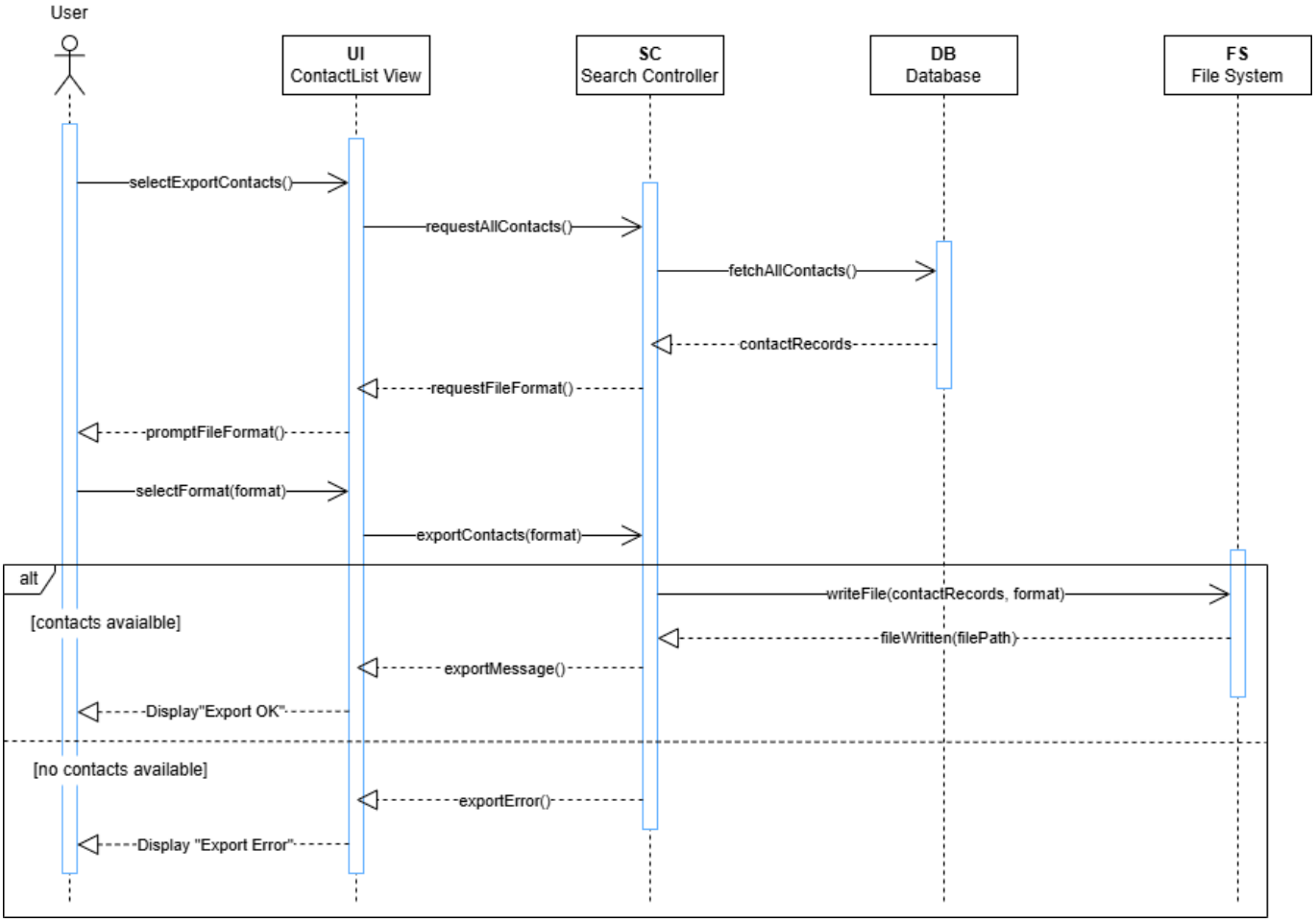
Sequence Diagram - Send SMS from Contact View



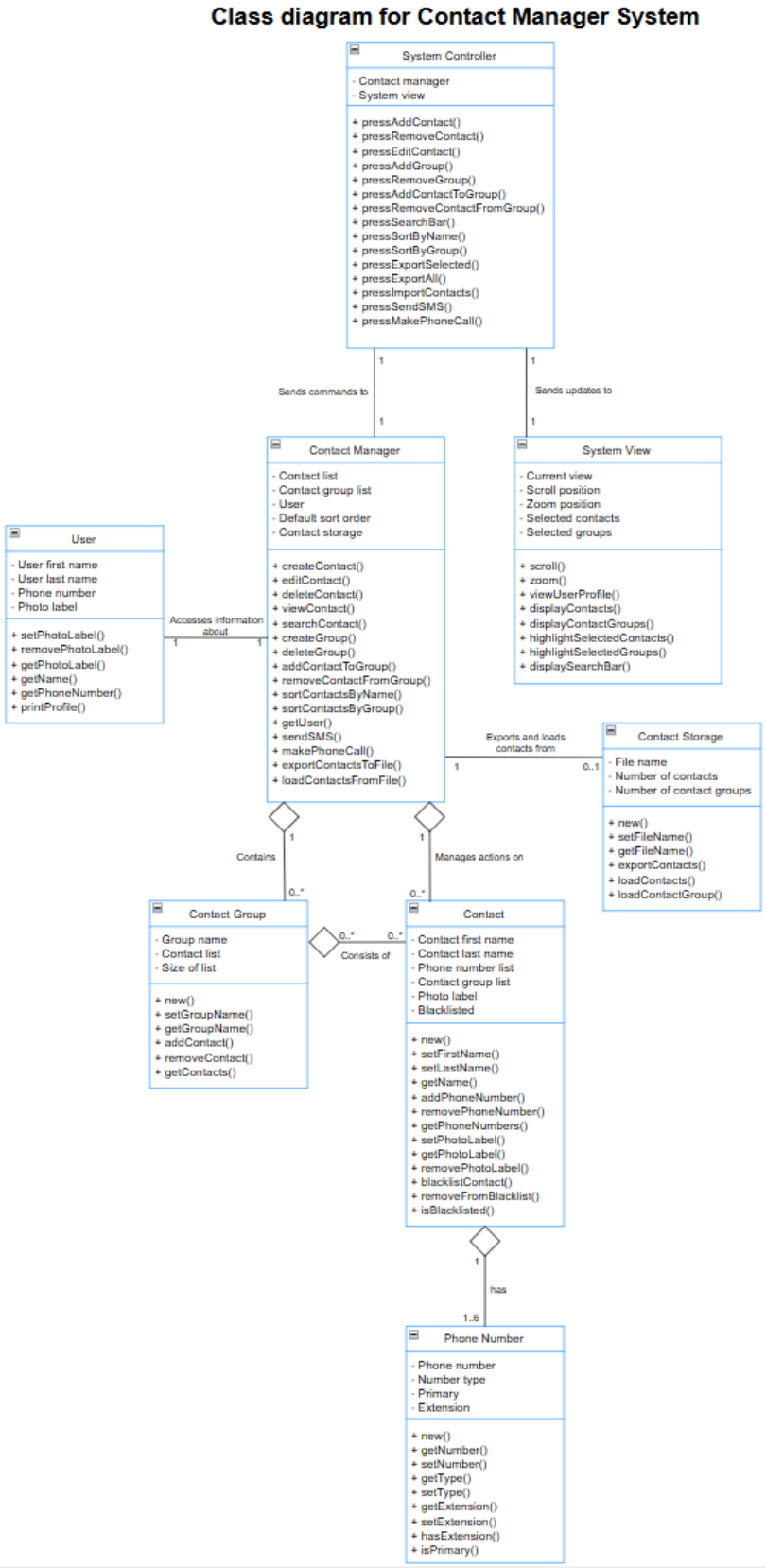
Sequence Diagram for Load Contacts



Sequence Diagram for Exporting Contacts



h. Class diagram



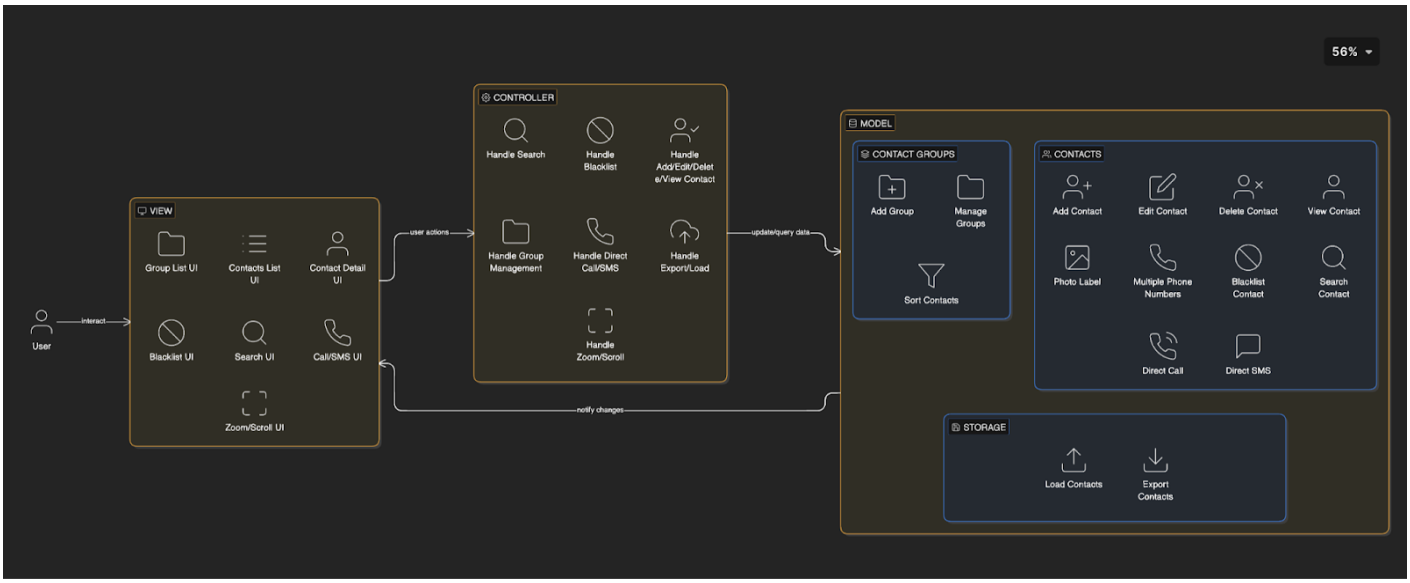
i. Architectural Design

When Used:

The model handles the data, for example storing and retrieving the contacts. The view is the thing the user sees and interacts with. The controller handles the logic, controls what the user sees, and actually handles user input. In the MVC pattern, the model also is separate from the view and controller. The controller updates the model according to user input (adding/deleting a contact).

Advantages:

This architectural pattern is actually useful to us because it is good at dealing with heavy user input. We will have a lot of user input (adding/deleting contacts, searching for contacts, etc.). Using MVC also allows us to make modifications to one part, such as the user interface or how the data is stored without necessarily having to redo the other parts as much. This is useful for the future if we ever decide to encrypt the contact or such.



4. Project Scheduling, Cost, Effort, and Pricing Estimation

The following estimations are calculated for the professional development of the Contact Manager System as a commercial-grade mobile application. These figures represent the project's actual implementation by a specialized software firm with the specified number of employees working on the project for the entire duration, rather than an academic effort.

a. Project Scheduling

- **Estimated Start Date**: May 4, 2026.
- **Estimated End Date**: October 16, 2026.
- **Justification**: The 24-week duration accounts for a standard Software Development Life Cycle (SDLC). It includes 3 weeks for requirement refinement and UI/UX design in

Figma, 14 weeks for the core MVC development and implementation (Model, View, Controller layers), 5 weeks for cross-platform QA testing on Android and iOS, and 2 weeks for deployment and client training. The allocated time should be enough to complete each phase and overcome any unexpected challenges that may arise for a professional development team.

- **Working Hours**: 8 working hours per day
- **Weekend Policy**: Weekends are not counted in the schedule to maintain professional labor standards and to ensure team productivity.

b. Cost, Effort, and Pricing Estimation

The Function Point modeling technique was chosen for estimation for this project as it can assess the complexity based on the software's specific capabilities, which is very helpful for this project. Furthermore, the software's capabilities align well into one of the Function Point categories, making it the suitable cost modeling method [1].

- **Unadjusted Function Point (UFP) Calculation**

The count is based on the functional requirements identified in the system design:

Functional Unit Category	Description	Count	Weight	Total Points
External Inputs (EI)	Add Contact, Edit Contact, Delete Contact, Import Contacts, Blacklist Contact, Manage Contact Groups	6	4 (Avg)	24
External Outputs (EO)	Export Contacts, Initiate Call, Send SMS	3	5 (Avg)	15
External Inquiries (EQ)	Search Contacts, Sort Contacts	2	4 (Avg)	8
Internal Logical Files (ILF)	Contact DB, Groups DB, Blacklist Registry	3	10 (Avg)	30
External Interface Files (EIF)	SMS Gateway API, Phone Network API, OS File System	3	7 (Avg)	21
Total UFP				98

- **Adjusted FP**: The Value Adjustment Factor (VAF) is 1.05 because of high usability, high mobile performance, frequent online data entry, moderate data communication with other systems, and high end-user efficiency requirements [1]. Multiplying unadjusted function point with this adjustment factor results in 102.9 Adjusted FP.
- **Effort Estimation**: Using a standard of 55 Lines of Code (LOC) per FP for mobile languages (Kotlin/Swift), the total codebase is approximately 5.7 KLOC [2].
    - COCOMO Organic Effort Formula:
      Effort = $2.4 \text{ X } (\text{KLOC})^{1.05}$
      $= 2.4 \text{ X } (5.7)^{1.05}$
      = approximately 14.8 person-months
- **Pricing**: Based on an average professional rate of $8,500/month for a mobile developer, the personnel cost is $125,800. Including a 20% overhead (facilities, management) and a 15% profit margin, the total price is $173,604 [3].

c. Estimated Cost of Hardware Products

- **Mobile Testing Suite**: $4,500 for a diverse range of flagship and mid-range Android and iOS devices to ensure compatibility [3].
- **On-Site Development Servers**: $2,000 for local database mirroring and version control hosting [3].
- **Total Hardware Cost**: $6,500

d. Estimated Cost of Software Products

- **Cloud Infrastructure**: $1,500/year for secure contact data storage and high-availability SMS gateway services [3].
- **Platform Licenses**: $300 (Apple Developer Program and Google Play Console fees).
- **Design & Prototyping Tools**: $600/year for enterprise-level Figma and automated testing tool licenses.
- **Total Software Cost**: $2,400

e. Estimated Cost of Personnel

- **Development Team**: 3 Mobile Developers at $8,500/month for 5 months = $127,500 [4]-[5].
- **Quality Assurance**: 1 QA Specialist for the final 2 months of the project = $12,000 [4]-[5].
- **Training & Documentation**: $2,500 for the creation of user manuals and training sessions for client administrative staff on backup recovery and blacklist management [4]-[5].
- **Total Personnel Cost**: $142,000

5. Software Test Plan

a. Test Plan

The unit that we will be testing for our project is the method isValidPhoneNumber(). The purpose of this test is to ensure that the method can accurately determine whether a given

phone number is valid or invalid. The expected behavior is that the method returns true if the provided phone number is valid and false otherwise. The provided parameter is the phone number in a String data type. Junit was used to test this method.

The method will verify phone number by checking whether it has been initialized, whether it includes any invalid characters, such as invalid letters and symbols, and whether it is of the acceptable length. This method will be tested with test cases that provide both valid and invalid phone numbers as inputs. The return type will be compared to determine correctness.

The scope of this test is to only validate the data logic of validating a phone number in. It is not concerned with displaying appropriate message to the user, updating the UI components, or other elements. It also only verifies that the phone number has acceptable digits and is of the appropriate length, and it is not concerned with verifying that the phone number currently exists. That aspect is left for the phone call system.

b. Code for One Unit

Unit:

```
public static boolean isValidPhoneNumber(String number) {
    if (number == null || number.trim().isEmpty()) return false;
    String cleaned = number.replaceAll("[\\s\\-().]+", "");
    return cleaned.matches("^\\+?[0-9]{7,15}$");
}
```

Test Cases:

```
@Test
public void isValidPhoneNumber_withDashes_returnsTrue() {
        assertTrue(InputValidator.isValidPhoneNumber("555-867-5309"));
}

@Test
public void isValidPhoneNumber_withParenthesesAndSpaces_returnsTrue() {
        assertTrue(InputValidator.isValidPhoneNumber("(202) 555-0147"));
}

@Test
public void isValidPhoneNumber_withDots_returnsTrue() {
        assertTrue(InputValidator.isValidPhoneNumber("555.867.5309"));
}

@Test
public void isValidPhoneNumber_internationalFormat_returnsTrue() {
        assertTrue(InputValidator.isValidPhoneNumber("+44 7911 123456"));
}
```

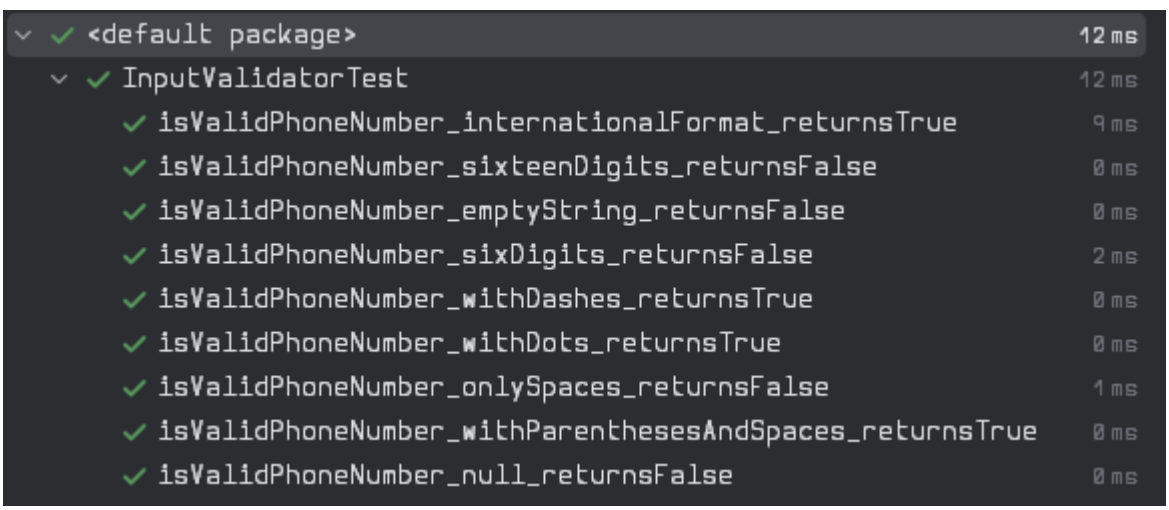
```
@Test
public void isValidPhoneNumber_null_returnsFalse() {
        assertFalse(InputValidator.isValidPhoneNumber(null));
}

@Test
public void isValidPhoneNumber_emptyString_returnsFalse() {
        assertFalse(InputValidator.isValidPhoneNumber(""));
}

@Test
public void isValidPhoneNumber_onlySpaces_returnsFalse() {
        assertFalse(InputValidator.isValidPhoneNumber("   "));
}

@Test
public void isValidPhoneNumber_sixDigits_returnsFalse() {
        assertFalse(InputValidator.isValidPhoneNumber("123456"));
}

@Test
public void isValidPhoneNumber_sixteenDigits_returnsFalse() {
        assertFalse(InputValidator.isValidPhoneNumber("1234567890123456"));
}
```



c. Test Case Explanation

The test cases covered all scenarios of possible valid and invalid phone numbers. The appropriate length for the phone number is 7 to 15 characters inclusive. And, the acceptable characters are digits 0-9, dash, space, parenthesis, dot, and plus sign for international country codes but only at the beginning.

For valid phone numbers, we had test case for appropriate length phone number with dashes. Then, we had a test case for appropriate length phone number with spaces, parentheses, and dashes. Then, we also test case with appropriate length phone number with dots. Then, there was test case with appropriate length phone number with plus sign (international country code). All of these cases returned true as expected. These test cases covered all possibilities of valid phone number scenarios as they checked digits and all the acceptable special characters, and they also checked the international country code scenario. All possible inputs with valid phone numbers should fall into one of these categories that were tested.

For invalid phone numbers, we had a test case with null or no phone number. Then, we had a test case where phone number did have a value, but it was an empty string. Then, we had another test case where phone number again had a value, but it was all spaces. Then, there was test case for phone number with acceptable characters, but only a length of 6. Finally, we had a test case for phone number with acceptable characters, but a length of 16. All of these test cases returned false as expected. These test cases covered possibilities of invalid phone number scenarios. The length requirement was ensured by checking the lower and upper point. Additionally, possible edge-cases related to phone numbers having acceptable length and acceptable characters, but no value, such as all spaces, have also been verified. All test cases passed successfully for this unit.

6. Comparison with Similar Designs

In order to achieve the standard requirements for our Contact Manager software, we studied numerous existing contact management applications, including Apple Contacts, contact applications on Android and Google. Throughout the research, the characteristics of these applications influenced us to take certain approaches for developing the design and interface for our project. Most of the contact manager applications shared many similarities in their features, such as providing capabilities to able to add, view, edit, and delete contacts. The information stored for each contact was also similar across these applications. They all could support multiple phone numbers per contact and provide ability to add a photo label for a contact. Other common features included being able to search contacts by name, the blacklist function, and ability to directly make phone calls and send SMS from a contact view. Unsurprisingly, the contact applications also support scrolling and zooming in and out as well [6]. Since these features were present in all these major contact manager applications, we also included these functionalities in the design of our software as well. Otherwise, our software would lack essential functionality in this domain and not have much appeal to the user.

In the organization aspect, being able to add contact groups, manage contacts groups, which includes add and remove functions, and then being able to sort them by name or group name were fairly similar among the researched applications [6]. Therefore, our system has been designed to iterate a similar pattern to give the user some sense of familiarity when navigating through our software.

However, there are some additional features in the researched application that are absent in our software. One such feature is that Apple, Android, and Google Contacts all allow user to send email through the contact management application, and have the option for sending group email to all contacts in the group. Another example is that both Apple and Android Contacts allow T9 dialing, which correlates numeric digits with English letters, therefore, allowing users to type by pressing single digits. This allows more rapid searching for a given contact name [7]. We have not implemented these features in our software as our aim is to first efficiently provide all the standard functionalities and then we may make improvements if deemed necessary later.

Another area where things differed was in the storage and data aspect of the contact list. For example, the Apple contacts application has the option for contacts information to be saved onto the cloud, allowing the user to have access to their contacts across platforms. On the other hand, our current design saves the user data locally, although it can be saved in the cloud in future updates. We are storing the data locally to avoid having security concerns and overcoming these concerns would require more resources and expertise from the development team.

Overall, it is evident that most of these contact management applications are very similar so that they can allow the user to have a familiarity when using the application. This is logical as this is a simple application with overlapping core features, so having similarity helps provide a user-friendly experience. Due to this, we have also made our software very similar to these researched applications, and we have left room for future improvement once core functionality is developed.

7. Conclusion

Overall, the development of the Contact Manager software was successful and built effectively with the work completed in the first deliverable providing the foundation. Our team was able to expand the initial system design by incorporating scheduling, cost estimation, and a structured test plan, which helped create a more complete and realistic representation of a professional software project. The use of an incremental process model allowed for flexibility and gradual improvement as our team progressed on the design and implementation of the project.

In this deliverable, some minimal changes were made compared to the original plan. Task delegation was adjusted to better distribute the workload and certain parts of the design and documentation were refined for clarity and consistency. In addition, the project was expanded to include more detailed planning components such as scheduling and estimating costs, which were not emphasized in our first deliverable. We also improvised our functional requirements by addressing the feedback provided in the first deliverable. We have now made the functional requirements a single, testable functionality. Other than these changes, our plan was realized without any obstacles or failures.

These changes were necessary to improve the overall quality of the project, our workflow, and ensured that we met the project requirements. Adjusting responsibilities helped maintain steady progress, while refining the design and adding realistic planning components made the project more accurate and complete.

In conclusion, this project strengthened the team's understanding of the full software development process by combining design, planning, estimation, and testing. We also gained knowledge about how to develop a prototype using Figma, and we also became aware of how useful a prototype is for understanding the initial design and interface of a software. The adjustments made throughout the project improved the final result and led to a more polished and practical project.

8. References

[1] International Function Point Users Group (IFPUG), *Function Point Counting Practices Manual (CPM) Release 4.3.1*, Princeton Junction, NJ: IFPUG, 2010.

[2] "Function Point Languages Table," QSM Research and Reference Library, [Online]. Available: https://www.qsm.com/resources/function-point-languages-table. [Accessed Mar. 27, 2026].

[3] B. Rizvi. "How Much Does It Cost to Develop an App in 2026?" Simpalm, Mar. 20, 2025. [Online]. Available: https://www.simpalm.com/blog/how-much-does-it-cost-to-make-an-app. [Accessed Mar. 28, 2026].

[4] A. Kolesnikova. "2023 Mid-Year Compensation Report," Levels.fyi, 2023. [Online]. Available: https://www.levels.fyi/blog/2023-mid-year-report.html. [Accessed Mar. 31, 2026].

[5] "Software Engineer Salaries – Mobile Developer," Glassdoor, 2024. [Online]. Available: https://www.glassdoor.com/Salaries/mobile-developer-salary-SRCH_KO0,16.htm. [Accessed Mar. 31, 2026].

[6] "Discover Hidden Features: Exploring the Depths of Your Phone's Contacts List," Ask, May 19, 2025. [Online]. Available: https://www.ask.com/news/discover-hidden-features-exploring-depths-phone-s-contacts-list. [Accessed Mar. 29, 2026].

[7] R. Agrawal. "With IOS 18, iPhone's Dialer Will Finally Get the Most Basic Smartphone Feature," AndroidAuthority, June 10, 2024. [Online]. Available: https://www.androidauthority.com/ios18-t9-dialing-3450399/. [Accessed Apr. 1, 2026].